

Problem Reconstruction: Conversational Memory for a Coding Assistant

Deliverable 1 - Problem Reconstruction

Domain: Coding / debugging assistant

How to read this document

This document reconstructs why conversational memory is a necessary component of an LLM-based assistant, by starting from the problem and tracing how simpler approaches fail. It does not name or describe the solution. It establishes the problem, the constraints, the approaches already tried, the evidence that each falls short, and the requirements that any credible system must therefore meet.

Detailed working is held in three companion files: the full failure cases in `failure_analysis.md`, the derived requirements in `first_principles.md`, and the causal chain in `historical_timeline.pdf`. This document summarises those and ties them together.

Claims are labelled by basis: **[SOURCE]** for a cited paper, **[INFERENCE]** for a conclusion I drew, **[FIRST-HAND]** for something I observed directly, and **[ASSUMPTION]** for anything taken as true without evidence.

1. Problem

An LLM-based assistant is stateless. Within a single session, its working context degrades and eventually exhausts a fixed token budget. Across sessions, it retains nothing. As a result, any task that spans more than a few turns, or more than one sitting, forces the user to carry the entire problem state themselves and to restate it whenever the session resets.

For a coding and debugging assistant specifically, this failure is acute. Debugging is inherently multi-session and multi-step: it depends on the user's stack, the versions in use, the constraints of the environment, and the approaches already ruled out. When none of this persists, the assistant cannot build on prior work, cannot avoid re-suggesting a fix the user already reported as failed, and cannot recognise when a fact it holds has been superseded.

The conditions under which the problem appears are therefore: conversations that exceed the context window, tasks that resume across sessions, and situations where earlier facts change or are corrected. All three are ordinary in real use.

2. Importance

The cost of leaving this unsolved is not limited to inconvenience. When a user must re-establish a complex problem on every session, the assistant can perform worse than the tool it was meant to replace.

I observed this directly. In early 2026, while building an end-to-end MLOps project, I spent roughly four to five days attempting to configure an Airflow DAG against a service that had recently moved from Azure into Microsoft Fabric. The assistant repeatedly produced configurations for the old arrangement, and each correction I supplied was lost as the context filled and the session had to restart. Re-supplying the full context in a fresh session reproduced the same loop. A conventional documentation search would have been faster than the assisted

process. **[FIRST-HAND]**

The broader consequences follow from this pattern. An assistant that loses state imposes repeated re-explanation cost on the user, produces incoherent behaviour when it contradicts what was established earlier, and never becomes more useful over time because it accumulates nothing. In a domain where the value of assistance depends on continuity, the absence of memory is not a missing feature but a cause of net-negative outcomes.

3. Constraints

Any solution operates within the following constraints. Each is stated with its basis.

Token budget. The context window is fixed and finite, and varies substantially by model, from roughly 2k to 200k tokens across common LLMs as of early 2024, corresponding to approximately 20 to 4,000 conversational turns under the source's stated assumptions [4, Table 1]. Widely used open-source models supported only a few dozen back-and-forth messages before exceeding their input limit [4, Introduction]. **[SOURCE]** For a coding assistant the effective turn count is lower, since pasted configuration, logs, and stack traces are more token-dense than ordinary prose. **[INFERENCE]**

Cost of extension. Enlarging the window is not a cheap remedy. Self-attention cost grows as $O(n^2 d)$ with sequence length [1, §3.2.1; 1, Table 1], so extending context incurs a quadratic increase in compute and memory [4, Introduction]. **[SOURCE]**

Correctness. An assistant that retains and reasserts a superseded fact is worse than one with no memory, because the error is confident and silent. When a retrieval index and the queried period were mismatched, accuracy fell to between 4 and 12 percent while the system continued to answer confidently [2, §4.5]. **[SOURCE]**

Latency. Memory that must be searched on every turn adds retrieval time to each response, so the retrieval method chosen constrains responsiveness. Approximate vector search exists precisely to keep such lookups within sub-second times at scale [4, §3.2.1]. **[SOURCE]** The system must therefore treat retrieval latency as a budgeted cost, not an afterthought. **[INFERENCE]**

Privacy. Debugging routinely involves pasting credentials, connection strings, and internal hostnames. Any information the system stores about a user must be protected, and sensitive material should not be admitted carelessly. None of the reviewed sources addresses this. **[INFERENCE]**

Multi-user isolation. Where a system serves more than one user, one user's memory must not surface in another's session. Retrieval ranked by content similarity provides no such separation, because different users' content can be genuinely similar. None of the reviewed sources addresses this. **[INFERENCE]**

The final two constraints, privacy and isolation, are not covered by any of the four source papers. They are derived from the demands of the domain and correspond to non-negotiable evaluation requirements.

4. Prior approaches

Five approaches have been used to give an assistant access to prior information. Each rests on an assumption that fails under identified conditions. The approaches are, in order of increasing sophistication:

1. **Stateless calls** - assume each request is self-contained.

2. **Full-context replay** - assume the whole history fits and is used.
3. **Human-as-memory** - assume the user will restate the problem on each reset.
4. **Summary-based memory** - assume a running summary preserves what matters.
5. **Naive retrieval** - assume the answer is among the top-K passages most similar to the query.

The full assumption and failure case for each is documented in `failure_analysis.md`. The evidence is summarised in the next section.

5. Failure evidence

Each approach fails for reasons that are cited or measured, not asserted. The key evidence, in brief:

Full-context replay fails on three independent grounds: the history does not fit a finite window [4, Table 1], enlarging the window is quadratically expensive [1, Table 1], and models under-use long context even when it fits [4, Introduction, citing Liu et al.]. Summary-based memory fails because summarisation loses the specifics later questions depend on: fixed-context baselines given a summary of five prior sessions answered only 32 to 39 percent of questions about those sessions correctly, and the more capable model did no better, indicating an architectural rather than a capability limit [4, Table 2]. Naive retrieval fails because the correct item is often outside the top-K [4, §3.2.1], a single pass cannot recover a miss or compose chained facts [4, §3.2.1; 4, §3.2.2], and ranking by similarity alone carries no sense of time or significance, so a superseded fact keeps winning [3, §4.1].

Two failures are constructed from first principles because no reviewed source addresses them, and both correspond to non-negotiable evaluation requirements. Cross-user leakage: similarity is computed over content, and different users' content can be genuinely similar, so nothing in similarity ranking prevents one user's private environment details from surfacing in another's session. Sensitive-data admission: in debugging, the useful information and the sensitive information are often the same, so an admission policy based on usefulness alone will preferentially retain secrets. Both are documented in full in `failure_analysis.md`.

The complete set of failure cases, with tags, is provided in the companion file. A summary table appears there.

6. Historical chain

The approaches did not appear independently. Each was a response to the bottleneck of the previous one. The chain, in brief:

The Transformer gave capable models with a fixed, expensive-to-extend window [1], which forced information out of the model and into an external store. Retrieval-augmented generation provided that store and solved knowledge access, but not memory management, since it ranks by similarity over a shared corpus [2]. Generative Agents added multi-signal ranking and synthesis, addressing what to rank and consolidate, but left retrieval misses, integrity, and isolation unsolved [3]. MemGPT reframed the limited context itself as a resource to be managed, but left unaddressed what is worth remembering about a user, along with forgetting, isolation, and sensitive data [4].

What remains unresolved across all four links is the conversational memory problem. The full chain, with the bottleneck at each step, is set out in `historical_timeline.pdf`, and is corroborated by MemGPT's own account of the

field's three lineages [4, §4].

7. Derived requirements

The failures above imply a set of properties that any credible system must have. Stated briefly, and organised by the question each serves:

What to remember: decide what to retain rather than storing everything; carry provenance; keep what the user reported distinct from what the model already believes.

What to forget: remove information completely on request; retire superseded facts so they stop influencing results.

How to retrieve: rank on more than similarity; retrieve iteratively rather than in a single pass; recognise when retrieval is incomplete rather than answer from a fragment.

How memory influences the conversation: fit within a bounded token budget; keep new statements consistent with prior ones from both user and assistant; personalise from accumulated memory; ensure a correction durably overrides the earlier fact.

How memory evolves: consolidate records rather than only accumulate them; update without retraining; persist across sessions.

Cross-cutting: isolate each user's memory; apply an admission policy that excludes sensitive data; defend stored memory against injected false content.

Each requirement is traced to the specific failure that produced it, with the reasoning for why no simpler alternative suffices, in `first_principles.md`. Note that these are required properties, not mechanisms. How they are achieved is the subject of the system design in Deliverable 4.

8. Open questions

The following uncertainties remain and will shape the design.

Deciding what is worth keeping. A selection policy is required, but the criteria for what to retain and what to discard are not settled by any reviewed source.

Making a correction stick. Retiring a superseded fact requires deciding which of two conflicting records is current, and what evidence should trigger that decision. No reviewed source addresses per-record supersession for a single user.

Knowing when retrieval is enough. Iterative retrieval is necessary but not sufficient. MemGPT reports that its own agent often stops searching before exhausting the store [4, §3.2.1]. How a system should decide it has searched enough, and when to report that it cannot find an answer rather than answer from incomplete evidence, is open. This is the citable form of a question raised by my own experience, where the assistant looped rather than reporting that it lacked the information. **[SOURCE + FIRST-HAND]**

Distinguishing a memory gap from a knowledge gap. My Airflow case involved both a memory failure and a genuine absence of documentation. A system should ideally tell the two apart, so that it reports missing knowledge rather than looping. How to make that distinction reliably is unresolved. **[INFERENCE]**

Weighting retrieval signals. Ranking on recency, importance, and relevance requires a weighting, and the source that introduced these signals notes the weighting could be improved by tuning [3, §8.2]. How the weights should be set,

and whether they should vary by domain, is open.

The boundary of integrity guarantees. Defending memory against injected false content is required in principle, but how far this can be guaranteed in a first implementation is uncertain, and may itself be only partially achievable.

References

- [1] A. Vaswani et al. *Attention Is All You Need*. 2017.
<https://arxiv.org/abs/1706.03762>
 - [2] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2020. <https://arxiv.org/abs/2005.11401>
 - [3] J. S. Park et al. *Generative Agents: Interactive Simulacra of Human Behavior*. 2023. <https://arxiv.org/abs/2304.03442>
 - [4] C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. 2023.
<https://arxiv.org/abs/2310.08560>
- Indirect source via [4]: N. F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. 2023. <https://arxiv.org/abs/2307.03172>